

Zero to Hero Study Handbook: startup-intelligence-graphrag

This handbook is built from static analysis of the repository files only. It is designed to help a new learner understand the system end-to-end, from code structure to runtime flow.

Module 1: Foundations & Architecture

1) What this project does

`startup-intelligence-graphrag` is a local-first GraphRAG system over SEC filings. It builds a corpus from a strict Hugging Face dataset, chunks filing text, embeds chunks, builds a graph from extracted entities/relationships, and evaluates multiple retrieval-generation techniques (dense, graph-aware, hybrid, agentic, and multimodal).

Primary use cases supported by the current code:

- Grounded Q&A over 10-K filings.
- Cross-company risk and strategy analysis.
- Retrieval technique benchmarking (`vector`, `graph_local`, `graph_global`, `hybrid`, `hybrid_sparse_dense`, `agentic_crag`, multimodal variants).
- Optional domain adaptation for generation (Unsloth + PEFT + TRL), isolated as an opt-in stage.

2) Core paradigms and patterns used in this repository

Definitions first, then where each appears:

- **Pipeline orchestration:** A fixed, ordered set of processing steps from raw data to artifacts.
 - Main implementations: `scripts/run_full_real_project.py`, `src/pipeline.py`, `scripts/run_complete_host.py`.
- **Dataclass-centric domain modeling:** Structured records for core entities.
 - Examples: `Filing`, `Chunk`, `RetrievedChunk`, `GenerationResult`, `JudgeScore`, `CommunitySummary`.
- **RAG (Retrieval-Augmented Generation):** Retrieve relevant context chunks, then generate grounded answers with citations.
 - Main flow: `src/generation.py`, `src/retrievers.py`.
- **GraphRAG:** Add graph structure (company, filing, section, entity nodes and relation/co-occurrence edges) to improve retrieval context.
 - Main flow: `src/graph.py`, graph-aware retrievers in `src/retrievers.py`.
- **Hybrid retrieval:** Fuse multiple retrieval channels.
 - RRF over dense + graph channels in `HybridRetriever`.
 - Dense + BM25 fusion in `HybridSparseDenseRetriever`.

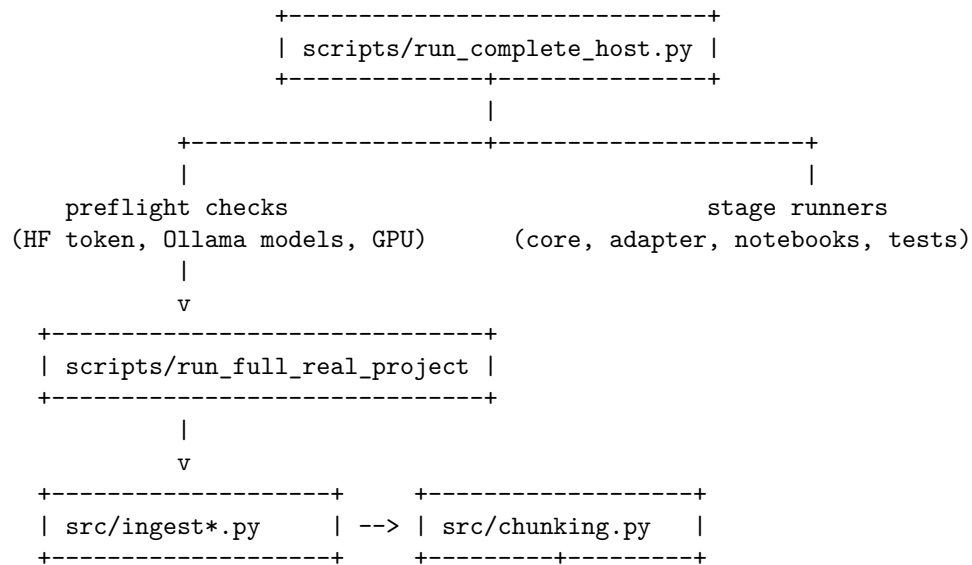
- **Agentic corrective loop (CRAG-style):** Classify query type, route retriever, grade retrieved chunks, retry with fallback retrievers.
 - Main class: `AgenticGraphRAG` in `src/agent.py`.
- **Multimodal extension pattern:** Add HTML table/figure text, OCR text, and vision text as retrievable units without changing base ingestion/retrieval modules.
 - Modules in `src/extensions`.
- **Extension isolation:** Optional components are placed in `src/extensions/` and separate scripts (for example, adapter stage).
 - Keeps base runtime path stable.

3) Architecture and interactions

High-level component map:

- **Configuration layer:** all runtime settings from `SIRAG_` env vars via `Settings`.
- **Data layer:** strict SEC ingestion (`deerfieldgreen/stk-sec-filings`) and filing normalization.
- **Retrieval substrate:** chunking + embeddings + FAISS + graph.
- **Retrieval strategies:** dense, graph-local, graph-global, hybrid, sparse+dense, multimodal.
- **Generation and evaluation:** answer generation, judge scoring, retrieval/generation/RAG metrics.
- **Orchestration layer:** project runners that sequence stages and write manifests/artifacts.

Text-based ASCII architecture:



File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/config.py	Central settings and project paths	<code>Settings</code> , <code>ensure_dirs()</code> , <code>as_dict()</code>	<code>dataset_repo</code> , <code>strict_dataset</code> , model names, chunk/eval/adapter settings
src/ingest.py	Generic strict ingestion and normalization	<code>Filing</code> , <code>build_corpus()</code> , <code>group_into_filings()</code>	<code>STRICT_DATASET_REPO</code> , <code>USEFUL_SECTIONS</code> , sentence bounds
src/ingest_deerfield.py	Deerfield dataset-specific adapter (current strict path)	<code>DeerfieldRecord</code> , <code>build_deerfield_corpus()</code>	<code>DEERFIELD_DATASET_ID</code> , <code>ITEMS_SECTION_MAP</code>
src/chunking.py	Token-aware chunk creation and persistence	<code>Chunk</code> , <code>chunk_corpus()</code> , <code>chunk_section()</code>	<code>chunk_size_tokens</code> , <code>chunk_overlap_tokens</code>
src/embeddings.py	Ollama embedding calls with cache	<code>embed_text()</code> , <code>embed_texts()</code> , <code>embedding_dim()</code>	<code>SETTINGS.embed_model</code> , <code>use_llm_cache</code>
src/vectorstore.py	FAISS index wrapper and dense search	<code>FaissVectorStore</code> , <code>SearchResult</code> , <code>build_from_chunks()</code>	<code>EMBEDDINGS_DIR</code> , <code>SETTINGS.embed_model</code>
src/extractor.py	LLM entity/relationship extraction	<code>Entity</code> , <code>Relationship</code> , <code>FilingExtraction</code> , <code>extract_from_filings()</code>	<code>ENTITY_TYPES</code> , <code>RELATION_TYPES</code> , extraction opts and limits
src/graph.py	Graph construction, communities, summaries	<code>build_graph()</code> , <code>detect_communities()</code> , <code>CommunitySummary</code> , <code>save_graph()</code>	<code>louvain_resolution</code> , graph node-id conventions
src/retrievers.py	Dense and graph-aware retrieval strategies	<code>RetrievedChunk</code> , <code>DenseVectorRetriever</code> , <code>GraphLocalRetriever</code> , <code>GraphGlobalRetriever</code> , <code>HybridRetriever</code>	fusion weights, <code>kerrf</code>
src/agent.py	Agentic CRAG-style orchestration	<code>AgenticGraphRAG</code> , <code>AgentResult</code> , <code>build_default_agent()</code>	<code>CLASSIFY_PROMPT</code> , <code>DOC_GRADE_PROMPT</code> , <code>max_iterations</code>
src/generation.py	Citation-grounded answer generation	<code>GenerationResult</code> , <code>build_context_block()</code> , <code>generate_answer()</code>	<code>SYSTEM_PROMPT</code> , <code>USER_PROMPT</code> , <code>generation_temperature</code>

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/evaluator.py	Retrieval and generation metric computation	<code>evaluate_retrieval()</code> , <code>evaluate_generation()</code> , metric helpers	<code>K()</code> , relevance in query rows
src/judge.py	LLM-as-a-judge scoring	<code>JudgeScore</code> , <code>judge_answer()</code> , <code>judge_generation()</code>	<code>judge_model</code> , <code>judge_temperature</code>
src/extensions/sparsebm25.py	BM25 sparse retrieval extension	<code>SparseBM25Retriever</code>	domain boost terms, BM25 scoring
src/extensions/hybridDenseBM25.py	Dense+BM25 fusion retriever	<code>HybridSparseDenseRetriever</code> , <code>HybridConfig</code>	<code>dense_weight</code> , <code>sparse_weight</code> , <code>rff_k</code>
src/extensions/multimodal.py	HTML table/figure-text unit extraction	<code>MultimodalUnit</code> , <code>build_multimodal_unit()</code>	modality tags: <code>table</code> , <code>figure_text</code>
src/extensions/multimodalOCR.py	OCR unit creation via Ollama command	<code>build_glm_ocr_command()</code> , <code>run_glm_ocr()</code>	<code>SIRAG_OCR_MODEL</code> , / <code>SETTINGS.ocr_model</code>
src/extensions/multimodalVision.py	Vision unit creation via Ollama chat	<code>run_qwen_vision()</code> , <code>build_vision_unit()</code>	<code>SIRAG_VISION_MODEL</code> , <code>SETTINGS.vision_model</code>
src/extensions/multimodalRetriever.py	Retrieval over multimodal units	<code>MultimodalRetriever</code>	modality weights, dense/sparse fusion ratio
src/extensions/rag_RAG.py	RAG level quality metrics	<code>RAGQualityScore</code> , <code>evaluate_rag_quality()</code>	<code>DEFAULT_EXTENSION_JUDGE_MODEL</code>
src/extensions/domain_adapt.py	Optimizing sloth+PEFT+TRL training and base-vs-adapter benchmark	<code>train_domain_adapter()</code> , <code>evaluate_base_vs_adapter()</code>	<code>adapter_from_chunks()</code> , <code>adapters</code> , GPU availability
src/multimodal_assets.py	Build multimodal source assets (HTML and image maps)	<code>fetch_filing_html_maps()</code> , <code>build_table_image_sources()</code> , <code>build_text_snapshot_images()</code>	<code>assets/multimodal/*</code>
scripts/run_full_real.py	Main project runtime path for full technique evaluation	<code>main()</code> , <code>_run_technique()</code> , <code>evaluate_agentic_endpoints()</code>	CLI args: <code>--n-companies</code> , <code>--endpoints</code> , ing/multimodal caps

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
scripts/run_complete.py	Single command host orchestrator with strict preflight and manifest	<code>_assert_hf_access_token()</code> , <code>_assert_ollama_model_exists()</code> , <code>_run_stage()</code>	<code>REQUIRED_OLLAMA_MODELS</code> , <code>OLLAMA_CONFIG</code> , HF token env
scripts/run_domain_adapter.py	Optional adapter-stage CLI	<code>run_placeholder_mode()</code> , <code>run_execute_mode()</code>	<code>mode</code> , <code>--force</code> , <code>--required</code>
scripts/run_pipeline.py	Simple placeholder or execute pipeline runner	<code>main()</code>	<code>--mode placeholder execute</code>
scripts/execute_notebooks.py	Backup notebooks and write executed copies	<code>main()</code>	<code>--timeout</code> , <code>--pattern</code> , output dir
tests	Contract-oriented safety rails for key behavior	contract tests per module	validates script contracts, model pinning, settings exposure

Module 3: Core Execution Flows

Flow A: Full real project execution (`scripts/run_full_real_project.py`)

Entrypoint: `main()`.

Step-by-step:

1. Parse CLI args (`--n-companies`, `--top-k`, chunk and multimodal limits).
2. Ensure artifact directories (`ensure_dirs()`).
3. Build corpus:
 - `build_corpus(...)` in `src/ingest.py`.
 - Under strict defaults, this dispatches to `build_deerfield_corpus(...)` in `src/ingest_deerfield.py`.
4. Build evaluation queries:
 - `build_eval_queries_from_filings(...)` then `save_eval_queries(...)`.
5. Build retrieval substrate:
 - `chunk_corpus(...)` and `save_chunks(...)`.
 - `build_from_chunks(...)` to create FAISS + metadata.
6. Build graph substrate:
 - `extract_from_filings(...)` -> `build_graph(...)` -> `detect_communities(...)` -> fast summaries -> `save_graph(...)`.
7. Run retrieval/evaluation techniques:

- Dense, graph local/global/hybrid, sparse+dense hybrid, agentic CRAG.
8. Run multimodal techniques:
 - `fetch_filing_html_map(...)`
 - `build_multimodal_units_from_html_map(...)`
 - image map from `build_table_image_sources(...)` or fallback `build_text_snapshot_images(...)`
 - OCR and vision units + unified v2 units.
 9. Persist outputs:
 - per-technique metrics and sample files
 - comparison figures
 - root compatibility placeholders
 - final `artifacts/run_summary.json`.

Core control snippet:

```
filings = build_corpus(...)
chunks = chunk_corpus(filings, ...)
store = build_from_chunks(chunks)
extractions = extract_from_filings(filings)
graph = build_graph(filings, extractions)
partition = detect_communities(graph)
```

Flow B: Single-query answer path (retrieve -> generate)

This logic is encapsulated by `answer_query(...)` in `src/generation.py`:

1. Call `retriever.retrieve(query, k=k)`.
2. Format retrieved chunks into numbered context and citation metadata (`build_context_block(...)`).
3. Call generator model with `SYSTEM_PROMPT + USER_PROMPT`.
4. Return `GenerationResult` and retrieved chunks.

Key output shape (`GenerationResult.to_dict()`):

```
{
  "query": "string",
  "answer": "string",
  "citations": [
    {
      "id": 1,
      "chunk_id": "string",
      "filing_id": "string",
      "ticker": "string",
      "company_name": "string",
      "section": "string",
      "score": 0.1234,
      "source": "vector|hybrid|...",
    }
  ]
}
```

```

        "via": "vector|graph_expand|community|...",
        "text_preview": "string"
    }
],
"model": "granite4.1:8b",
"prompt_tokens": 0,
"completion_tokens": 0
}

```

Flow C: Agentic CRAG loop (src/agent.py)

Entrypoint: `AgenticGraphRAG.run(query, k)`.

Steps:

1. `classify_query(...)` -> local, global, or factual.
2. Select retriever priority order (`_primary_retriever_order(...)`).
3. Iteratively retrieve and grade:
 - retrieve chunks with current retriever
 - grade chunk relevance via LLM JSON (`_grade_chunks(...)`)
 - if enough relevant chunks, stop; else fallback to next retriever.
4. Generate final answer with `generate_answer(...)`.
5. Return `AgentResult` with trace steps.

Minimal trace semantics:

`classify -> retrieve(iteration N, retriever X) -> grade_docs -> ... -> generate`

Flow D: Multimodal execution path

Main modules:

- Asset preparation: `src/multimodal_assets.py`
- Unit extraction: `src/extensions/multimodal.py`, `src/extensions/multimodal_ocr.py`, `src/extensions/multimodal_vision.py`
- Retrieval: `src/extensions/multimodal_retriever.py`

Steps:

1. Build `filing_id` -> html map (`fetch_filing_html_map`).
2. Extract table and figure text `MultimodalUnits` from HTML.
3. Build image sources for OCR/vision from tables or fallback text snapshots.
4. OCR channel (`run_glm_ocr`) and vision channel (`run_qwen_vision`).
5. Retrieve via multimodal dense+sparse fusion with modality weights.

Key input/output data structures you should know

Filing object (src/ingest.py)

```

Filing(
    company_name, cik, ticker, exchange, state_of_incorporation, sic,

```



```

    form, filing_date, report_date, filing_id, source_split, source_row_count,
    sections: dict[str, list[str]]
)

```

Chunk object (src/chunking.py)

```

Chunk(
    chunk_id, filing_id, ticker, company_name, section,
    text, sentence_ids: list[int], token_count: int
)

```

RetrievedChunk object (src/retrievers.py)

```

RetrievedChunk(
    chunk_id, filing_id, ticker, company_name, section, text, sentence_ids,
    score, source, graph_score, vector_score, via, metadata
)

```

Eval query row (JSONL in data/eval/eval_queries.jsonl)

```

{
  "query_id": "q01_local_business_es",
  "query": "...",
  "query_type": "local|global|factual",
  "relevant_tickers": ["ES"],
  "relevant_sections": ["Business", "Management's Discussion and Analysis (MD&A)"],
  "keyword_hints": ["..."],
  "reference_answer": "...",
  "retrieval_relevant_chunk_ids": []
}

```

Run summary (artifacts/run_summary.json) Top-level keys include:

- mode, timestamp_utc, started_utc, duration_seconds
- dataset_repo
- n_filings, n_chunks, n_eval_queries, n_graph_nodes, n_graph_edges
- n_multimodal_units, n_ocr_units, n_vision_units
- settings, schema_snapshot, techniques, figures

Module 4: Setup & Run Guide

1) Clean-machine setup

This repository expects uv workflows and Python 3.12.10.

```

cd /home/ahmad/AI/Github/startup-intelligence-graphrag
uv python install 3.12.10
uv venv --python 3.12.10 .venv

```

```
source .venv/bin/activate
uv sync
```

Optional adapter dependencies:

```
uv sync --extra adapter
```

2) Required external services

- **Ollama** must be running and models must be available:

```
ollama pull qwen3-embedding:4b
ollama pull granite4.1:8b
ollama pull glm-ocr:latest
ollama pull qwen3.5:4b
```

- **Hugging Face token** is required for strict host run preflight:
 - HUGGINGFACE_HUB_TOKEN or HF_TOKEN (checked in `scripts/run_complete_host.py`).

3) Environment configuration

Recommended:

1. Copy `.env.example` to `.env`.
2. Keep strict dataset defaults unless intentionally changing behavior.

Key env vars from `.env.example`:

- Dataset and strictness:
 - SIRAG_DATASET_REPO
 - SIRAG_DATASET_CONFIG
 - SIRAG_DATASET_SPLITS
 - SIRAG_STRICT_DATASET
- Local model endpoints and model IDs:
 - SIRAG_OLLAMA_HOST
 - SIRAG_EMBED_MODEL
 - SIRAG_GENERATOR_MODEL
 - SIRAG_JUDGE_MODEL
 - SIRAG_EXTENSION_JUDGE_MODEL
 - SIRAG_OCR_MODEL
 - SIRAG_VISION_MODEL
- Optional adapter controls:
 - SIRAG_ADAPTER_ENABLE
 - SIRAG_ADAPTER_BASE_MODEL
 - SIRAG_ADAPTER_USE_4BIT
 - SIRAG_ADAPTER_MAX_SEQ_LENGTH
 - SIRAG_ADAPTER_MAX_TRAIN_EXAMPLES
 - SIRAG_ADAPTER_MAX_EVAL_EXAMPLES
 - SIRAG_ADAPTER_TRAIN_SPLIT
 - SIRAG_ADAPTER_LORA_R

- SIRAG_ADAPTER_LORA_ALPHA
- SIRAG_ADAPTER_LORA_DROPOUT
- SIRAG_ADAPTER_LEARNING_RATE
- SIRAG_ADAPTER_PER_DEVICE_BATCH_SIZE
- SIRAG_ADAPTER_GRADIENT_ACCUMULATION_STEPS
- SIRAG_ADAPTER_MAX_STEPS
- SIRAG_ADAPTER_EVAL_STEPS
- SIRAG_ADAPTER_SAVE_STEPS
- SIRAG_ADAPTER_LOG_STEPS
- SIRAG_ADAPTER_MAX_NEW_TOKENS
- Runtime sizing:
 - SIRAG_DEFAULT_N_COMPANIES
 - SIRAG_PLACEHOLDER_MODE

4) Typical command sequences

Core real pipeline:

```
source .venv/bin/activate
python scripts/run_full_real_project.py --n-companies 30 --force-download
```

Strict host orchestration (core + adapter + notebooks + tests + completion manifest):

```
source .venv/bin/activate
python scripts/run_complete_host.py --profile comprehensive --force-download
```

Optional adapter stage only:

```
source .venv/bin/activate
export SIRAG_ADAPTER_ENABLE=true
python scripts/run_domain_adapter.py --mode execute --n-companies 30 --force
```

Notebook execution:

```
source .venv/bin/activate
python scripts/execute_notebooks.py --timeout 3600
```

Placeholder seeding path:

```
source .venv/bin/activate
python scripts/run_pipeline.py --mode placeholder
python scripts/seed_extension_placeholders.py
```

5) Migration/seeding steps for databases or external services

- There is **no relational database layer** and no migration framework in this repository.
- Artifact seeding is file-based:
 - Base placeholders: `src/pipeline.py` -> `seed_placeholder_artifacts()`
 - Extension placeholders: `scripts/seed_extension_placeholders.py`

- Domain adapter placeholders: `src/extensions/domain_adapter.py`
-> `seed_domain_adapter_placeholders()`

Module 5: Study Plan & Practice Exercises

Ordered self-study plan

Use this reading order:

1. `README.md` and `pyproject.toml` to understand project scope, dependencies, and run commands.
2. `src/config.py` to learn the settings contract and artifact path layout.
3. Ingestion and corpus shaping:
 - `src/ingest.py`
 - `src/ingest_deerfield.py`
4. Retrieval substrate:
 - `src/chunking.py`
 - `src/embeddings.py`
 - `src/vectorstore.py`
5. Graph and retrieval logic:
 - `src/extractor.py`
 - `src/graph.py`
 - `src/retrievers.py`
6. Generation, judging, and metrics:
 - `src/generation.py`
 - `src/judge.py`
 - `src/evaluator.py`
 - `src/extensions/rag_metrics.py`
7. Runtime orchestration:
 - `scripts/run_full_real_project.py`
 - `scripts/run_complete_host.py`
 - `scripts/run_domain_adapter.py`
8. Optional techniques:
 - sparse/hybrid extensions
 - multimodal extensions
 - domain adapter module
9. Finally, use `tests/` as behavior contracts to validate your understanding of interfaces.

Practice exercises (with solution outlines)

Exercise 1 Question: In this repository, what exact function chain turns raw SEC data into `Filing` objects under strict defaults?

Solution outline: - Start at `build_corpus()` in `src/ingest.py`. - Because strict repo defaults to deerfield, it delegates to `build_deerfield_corpus()` in `src/ingest_deerfield.py`. - Deerfield flow: `download_deerfield_rows()`

```
-> _record_from_row() -> _to_filing() -> _sample_filings() ->
save_deerfield_artifacts().
```

Exercise 2 Question: Where is chunk overlap implemented, and is it sentence-based or token-based?

Solution outline: - See `chunk_section()` in `src/chunking.py`. - Overlap is token-based (`overlap_tokens`) while moving sentence boundaries. - `tokenized` sentence arrays are used to walk and backtrack starts.

Exercise 3 Question: Which object fields are required for retrieval evaluation relevance derivation?

Solution outline: - Read `_derive_relevant_ids()` in `src/evaluator.py`.
- Query-side fields: `retrieval_relevant_chunk_ids`, `relevant_tickers`, `relevant_sections`, `keyword_hints`. - Metadata-side fields: `chunk_id`, `ticker`, `section`, `text`.

Exercise 4 Question: How does `GraphLocalRetriever` compute graph expansion influence?

Solution outline: - In `GraphLocalRetriever._expand_sections()`, seed sections map to neighboring entity nodes. - Entity boost is scaled by inverse sqrt degree. - Section contribution uses edge weight and seed score. - Section scores are converted to chunk-level graph scores via `_section_to_chunks`.

Exercise 5 Question: Explain the exact difference between `GraphGlobalRetriever` and `GraphLocalRetriever`.

Solution outline: - Local retriever expands from seed chunks through entity hops. - Global retriever embeds community summaries and scores communities directly against query embedding, then collects chunks via section neighbors. - Local is query-neighborhood expansion; global is community-theme routing.

Exercise 6 Question: In the agentic flow, what causes retriever switching?

Solution outline: - In `AgenticGraphRAG.run()`, each iteration grades chunks via `_grade_chunks()`. - If filtered relevant chunks are below threshold (`min(2, k)`), it falls back to next retriever in `_primary_retriever_order(query_type)`.

Exercise 7 Question: Track how multimodal image assets are generated when HTML table extraction fails.

Solution outline: - In `scripts/run_full_real_project.py`, `build_table_image_sources(...)` is called first. - If `image_map` is empty, fallback is `build_text_snapshot_images(...)`. - OCR/vision units are then built from this fallback `image_map`.

Exercise 8 Question: What files prove a strict host run is complete vs failed?

Solution outline: - `artifacts/run_completion_manifest.json` is the authority. - `status == "completed"` indicates full pass. - In current artifacts, status is "failed" with failure message on `core_pipeline`.

Understanding Checklist

Use this to self-verify:

- Can you explain how `build_corpus()` reaches `build_deerfield_corpus()` under current strict settings?
- Can you describe how `Chunk` and `RetrievedChunk` differ, and why both are needed?
- Can you trace one query through `HybridRetriever` and into `generate_answer()` with citations?
- Can you explain node and edge types added by `build_graph()` and why section provenance edges exist?
- Can you explain the agentic retry condition in `AgenticGraphRAG.run()`?
- Can you list which files are written by `run_full_real_project.py` vs `run_complete_host.py`?
- Can you identify all required runtime dependencies (HF token + Ollama models) before strict host orchestration?
- Can you explain where multimodal channels are additive (not replacing core text pipeline)?